

HDOC Day-33

Associative Properties of Matrix Addition and Multiplication Using Variable-Length 2D Arrays

Aim

To write a menu-driven C program using separate functions and variable-length 2D integer arrays to verify the associative property of matrix addition and matrix multiplication.

Algorithm

1. Start the program and display a menu: (1) Matrix Addition Associativity, (2) Matrix Multiplication Associativity, (3) Exit.
2. Accept the user choice and use a switch statement to call the required function.
3. For matrix addition, read dimensions m and n, then read three matrices A, B and C of size m x n.
4. Compute $(A + B) + C$ and $A + (B + C)$, display both matrices, and compare corresponding elements.
5. If all corresponding elements are equal, report that matrix addition is associative.
6. For matrix multiplication, read dimensions so that A is m x n, B is n x p and C is p x q.
7. Compute $A \times B$, then $(A \times B) \times C$. Separately compute $B \times C$, then $A \times (B \times C)$.
8. Display both final matrices and compare corresponding elements.
9. If they are equal, report that matrix multiplication is associative.
10. Continue displaying the menu until Exit is selected. Display an error for an invalid choice.

C Program

```
#include <stdio.h>

void additionAssociative(void);
void multiplicationAssociative(void);

int main(void)
{
    int choice;
    do {
        printf("\n----- MATRIX ASSOCIATIVITY MENU -----\n");
        printf("1. Verify Associativity of Matrix Addition\n");
        printf("2. Verify Associativity of Matrix Multiplication\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: additionAssociative(); break;
            case 2: multiplicationAssociative(); break;
            case 3: printf("Exiting program...\n"); break;
            default: printf("Invalid choice! Please enter 1 to 3.\n");
        }
    } while (choice != 3);
    return 0;
}

void additionAssociative(void)
{
    int m, n, i, j, same = 1;
    printf("Enter rows and columns (m n): ");
    scanf("%d %d", &m, &n);
    int A[m][n], B[m][n], C[m][n];
    int left[m][n], right[m][n];

    printf("Enter matrix A:\n");
```

```

for (i=0;i<m;i++) for (j=0;j<n;j++) scanf("%d", &A[i][j]);
printf("Enter matrix B:\n");
for (i=0;i<m;i++) for (j=0;j<n;j++) scanf("%d", &B[i][j]);
printf("Enter matrix C:\n");
for (i=0;i<m;i++) for (j=0;j<n;j++) scanf("%d", &C[i][j]);

for (i=0;i<m;i++)
    for (j=0;j<n;j++) {
        left[i][j] = (A[i][j] + B[i][j]) + C[i][j];
        right[i][j] = A[i][j] + (B[i][j] + C[i][j]);
        if (left[i][j] != right[i][j]) same = 0;
    }

printf("\n(A+B)+C:\n");
for (i=0;i<m;i++) { for (j=0;j<n;j++) printf("%d ", left[i][j]); printf("\n"); }
printf("A+(B+C):\n");
for (i=0;i<m;i++) { for (j=0;j<n;j++) printf("%d ", right[i][j]); printf("\n"); }

if (same) printf("Matrix addition is associative.\n");
else printf("Matrix addition is not associative.\n");
}

void multiplicationAssociative(void)
{
    int m,n,p,q,i,j,k,same=1;
    printf("Enter m n p q: ");
    scanf("%d %d %d %d", &m,&n,&p,&q);
    int A[m][n], B[n][p], C[p][q];
    int AB[m][p], BC[n][q], left[m][q], right[m][q];

    printf("Enter matrix A (%d x %d):\n",m,n);
    for(i=0;i<m;i++) for(j=0;j<n;j++) scanf("%d",&A[i][j]);
    printf("Enter matrix B (%d x %d):\n",n,p);
    for(i=0;i<n;i++) for(j=0;j<p;j++) scanf("%d",&B[i][j]);
    printf("Enter matrix C (%d x %d):\n",p,q);
    for(i=0;i<p;i++) for(j=0;j<q;j++) scanf("%d",&C[i][j]);

    for(i=0;i<m;i++) for(j=0;j<p;j++) {
        AB[i][j]=0; for(k=0;k<n;k++) AB[i][j]+=A[i][k]*B[k][j];
    }
    for(i=0;i<m;i++) for(j=0;j<q;j++) {
        left[i][j]=0; for(k=0;k<p;k++) left[i][j]+=AB[i][k]*C[k][j];
    }
    for(i=0;i<n;i++) for(j=0;j<q;j++) {
        BC[i][j]=0; for(k=0;k<p;k++) BC[i][j]+=B[i][k]*C[k][j];
    }
    for(i=0;i<m;i++) for(j=0;j<q;j++) {
        right[i][j]=0; for(k=0;k<n;k++) right[i][j]+=A[i][k]*BC[k][j];
        if(left[i][j]!=right[i][j]) same=0;
    }

    printf("\n(AxB)xC:\n");
    for(i=0;i<m;i++){ for(j=0;j<q;j++) printf("%d ",left[i][j]); printf("\n"); }
    printf("Ax(BxC):\n");
    for(i=0;i<m;i++){ for(j=0;j<q;j++) printf("%d ",right[i][j]); printf("\n"); }

    if(same) printf("Matrix multiplication is associative.\n");
    else printf("Matrix multiplication is not associative.\n");
}

```

Test Case Table

Test	Choice	Input	Expected Result
1	1	m=2, n=2; A=[[1,2],[3,4]], B=[[5,6],[7,8]], C=[[1,1],[1,1]]	Both sides = [[7,9],[11,13]]; addition is associative
2	1	m=1, n=3; A=[1 2 3], B=[4 5 6], C=[7 8 9]	Both sides = [12 15 18]; addition is associative
3	2	m=n=p=q=2; A=[[1,2],[3,4]], B=[[2,0],[1,2]], C=[[1,1],[0,1]]	(AxB)xC = Ax(BxC); multiplication is associative
4	4	-	Invalid choice message
5	3	-	Program exits

Result

The menu-driven C program was successfully designed using separate functions and variable-length 2D arrays. It verifies that matrix addition satisfies $(A+B)+C = A+(B+C)$ for matrices of the same dimensions, and that compatible matrix multiplication satisfies $(A \times B) \times C = A \times (B \times C)$. The program also handles menu selection and exits when requested.